Transformers

Tensorflow and Huggingface

Paulo Vieira

Agenda

Tensorflow (tf)

Huggingface (hf)

Transformers em tf

Transformer no hf

Os transformers

Transformers são um tipo de arquitetura de redes neurais usada para processar sequências, como texto, áudio ou até imagens. Foram introduzidos no paper **“Attention Is All You Need” (2017)**.

Em vez de ler palavra a palavra (como RNNs), o Transformer: olha para **toda a sequência ao mesmo tempo** e decide o que é importante

O operador core: Attention

O componente principal é o **self-attention**:

- cada palavra “olha” para todas as outras
- decide quais são mais relevantes

Exemplo:

“O gato que estava cansado dormiu”

“dormiu” pode ligar-se mais a “gato” do que a outras palavras numa Head de Attention

Componentes principais

i. Self-Attention (Q, K, V)

- Q → pergunta
- K → contexto
- V → informação

o modelo decide “a quem prestar atenção”

ii. Multi-Head Attention: várias atenções em paralelo

- uma aprende gramática
- outra contexto
- outra relações longas

iii. Feed-Forward (FFN): processa cada palavra individualmente após a atenção

iv. Positional Encoding: dá noção de ordem (porque o modelo vê tudo ao mesmo tempo)

Tipos de transformers

Tipos de Transformers

Encoder (ex: BERT)
- entende texto
- Exemplo de uso; usado para classificação

Decoder (ex: GPT)
- gera texto
- Exemplo de uso; usado para chat

Encoder-Decoder (ex: BART)
- transforma texto → texto
- Exemplo de uso; usado para tradução, resumo

Porque são importantes
Transformers revolucionaram a IA porque:
- permitem treino paralelo (rápido)
- captam relações longas no texto
- escalam muito bem (modelos gigantes)

Resumo
Transformers são:
- uma **arquitetura de redes neurais baseada em attention**
- usada para **processar e gerar sequências**
- base de modelos como:
  - ChatGPT
  - BERT
  - BART

Transformers, na livraria transformer do HuggingFace

Principais modelos Transformer (Hugging Face)					
Modelo	Tipo	Encoder	Decoder	Max tokens (aprox.)	Uso principal
BERT	Encoder	S	N	512	compreensão
DistilBERT	Encoder	S	N	512	versão leve do BERT
RoBERTa	Encoder	S	N	512	BERT melhorado
ALBERT	Encoder	S	N	512	BERT eficiente
GPT-2	Decoder	N	S	1024	geração de texto
GPT-Neo	Decoder	N	S	2048	GPT open-source
GPT-J	Decoder	N	S	2048	geração avançada
T5	Encoder-Decoder	S	S	512–1024	transformação de texto
BART	Encoder-Decoder	S	S	1024	resumo / geração
MarianMT	Encoder-Decoder	S	S	~512	tradução
Pegasus	Encoder-Decoder	S	S	1024	resumo
Longformer	Encoder	S	N	4096+	textos longos
BigBird	Encoder	S	N	4096–8192	documentos longos

Attention vs self-attention

1. **Attention** (geral)
2. **Self-attention** (caso especial de attention)

1. Attention (geral)

Liga **duas sequências diferentes**

Exemplo:

- Input A → pergunta
- Input B → contexto

Q ← sequência A

K, V ← sequência B

O modelo pergunta:

"Onde devo olhar na outra sequência?"

Exemplo real: No decoder de um Transformer:

- Q → frase que está a ser gerada
- K, V → output do encoder

isto chama-se **cross-attention** (olhar output para input, tokens do decoder para os tokens do encoder)

K,V

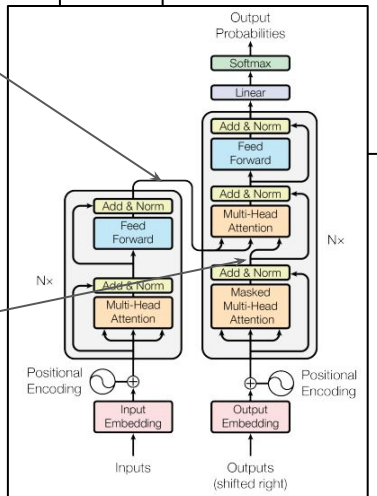

Figure 1: The Transformer - model architecture.

2. Self-Attention

Liga **uma sequência consigo própria**

Q = K = V = mesma sequência

O modelo pergunta:

"Que palavras desta frase são importantes entre si?"

Exemplo

Frase:

"O gato que estava cansado dormiu"

"dormiu" numa head attention pode atenção a "gato"

Attention vs self-attention

Diferença direta				
Tipo	Q	K	V	Uso
Attention	diferente	diferente	diferente	entre sequências
Self-attention	igual	igual	igual	dentro da mesma sequência

Intuição simples

- **Self-attention** → olhar para si próprio
- **Attention (geral)** → olhar para outra coisa

Self-attention é attention aplicada à própria sequência.

Resumo

- Attention = mecanismo geral
- Self-attention = caso específico (mais usado nos Transformers)

Attention vs self-attention

```python
import tensorflow as tf
# queries (ex: decoder)
query = tf.random.normal((1, 3, 4))
# keys/values (ex: encoder)
value = tf.random.normal((1, 5, 4))
print("query=",query)
print("value=",value)
# print(attention.trainable_variables)
# o operador de attention por default não usa scaling
attention = tf.keras.layers.Attention()
print(attention)
output = attention([query, value])
print(output.shape)
print(output)
```

A classe Attention() não usa scaling for default. Se nada foi configurado não esta a executar scaling

```
query= tf.Tensor(
[[[-0.38622034 -2.2790008  -1.3204155   0.26023272]
  [-0.30173767  1.0758208  -1.3431305   1.7904155 ]
  [ 0.62751806 -1.6729915  -0.7631812  -1.207677  ]]], shape=(1, 3, 4), dtype=float32)
value= tf.Tensor(
[[[ 0.89591265  1.0653707  -0.40023652  1.1784804 ]
  [-1.2498264   0.4218687  -0.3573873  -0.5441085 ]
  [-1.4230695  -1.1641879  -1.3713232   0.5767034 ]
  [ 0.13055043 -1.0528728  -0.5664507   1.1693414 ]
  [ 0.00183201  0.3569577  -0.67151463  0.97809213]]], shape=(1, 5, 4), dtype=float32)
<Attention name=attention_1, built=False>
(1, 3, 4)
tf.Tensor(
[[[-1.1862143  -1.1297388  -1.2451888   0.660967  ]
  [ 0.26397404  0.42465356 -0.602854    1.0151454 ]
  [-0.82688785 -0.918865   -0.9955721   0.7033558 ]]], shape=(1, 3, 4), dtype=float32)
```

SELF-ATTENTION

Vamos usar **duas palavras**:

$x_1 = [1, 0]$ (palavra 1)

$x_2 = [0, 1]$ (palavra 2)

E fazer:

Q = K = V = x

Matriz Q, K, V

Q = K = V =[[1, 0],[0, 1]]

Passo 1: QK^T

QK^T=[1,0] · [1,0]=1[1,0] · [0,1]=0[0,1] · [1,0]=0[0,1] · [0,1]=1

QK^T = [1,0]·[1,0] = 1 [1,0]·[0,1] = 0 [0,1]·[1,0] = 0 [0,1]·[0,1] = 1

QK^T=[1,0] · [1,0]=1[1,0] · [0,1]=0[0,1] · [1,0]=0[0,1] · [0,1]=1

Resultado:

[1, 0

0, 1]

Passo 2: softmax por linha

[1, 0] → [0.73, 0.27]

[0, 1] → [0.27, 0.73]

Passo 3: multiplicar por V

$output_1 = 0.73 \cdot x_1 + 0.27 \cdot x_2$

$output_2 = 0.27 \cdot x_1 + 0.73 \cdot x_2$

Attention vs self-attention

A classe MultiHeadAttention usa scaling por default

```
query= tf.Tensor(
[[[-0.59181523 -0.7115664  -2.7899914  -1.3646775 ]
  [ 0.40614307  2.1979733   0.9502217  -1.29784   ]
  [-0.7291663   0.9334245   1.519525   -2.3075078 ]]], shape=(1, 3, 4), dtype=float32)
key= tf.Tensor(
[[[ 2.929228   -0.9075119  -0.7065092   0.46491373]
  [-0.04893224  0.913724   -0.3800269   0.30726114]
  [ 1.9199867   0.27090794  0.5308807   0.05034263]
  [-0.08056321  1.4665537   2.0803664   1.6693518 ]
  [ 0.23826072  0.71721023  0.2368926  -0.55783767]]], shape=(1, 5, 4), dtype=float32)
value= tf.Tensor(
[[[ 0.14276342  0.7517144   1.2521734  -0.10438479]
  [ 0.66062653 -0.29512024  0.63977396 -0.37872958]
  [-1.3188046  -0.4572186  -1.9034461   0.4621055 ]
  [-0.3314979  -0.02035132 -0.29541653  0.95787543]
  [-0.7323571  -0.0797712  -1.6417551   1.0093889 ]]], shape=(1, 5, 4), dtype=float32)
Output shape: (1, 3, 4)
Scores shape: (1, 2, 3, 5)

Head 0
tf.Tensor(
[[0.02946192 0.15390117 0.11876638 0.46875313 0.22911742]
 [0.1635451  0.12814203 0.19625235 0.35915577 0.15290473]
 [0.14767838 0.13810715 0.19831415 0.3516651  0.16423526]], shape=(3, 5), dtype=float32)

Head 1
tf.Tensor(
[[0.41161644 0.139666   0.23476566 0.04637492 0.167577  ]
 [0.12778671 0.1992387  0.14418638 0.34527385 0.1835143 ]
 [0.07225978 0.18521813 0.12140637 0.43034822 0.19076748]], shape=(3, 5), dtype=float32)
```

Attention

```python
import tensorflow as tf

mha = tf.keras.layers.MultiHeadAttention(
    num_heads=2,
    key_dim=4
)

query = tf.random.normal((1, 3, 4))
key = tf.random.normal((1, 5, 4))
value = tf.random.normal((1, 5, 4))

print("query=", query)
print("key=", key)
print("value=", value)

# obter também scores
output, scores = mha(
    query=query,
    key=key,
    value=value,
    return_attention_scores=True
)

print("Output shape:", output.shape)
print("Scores shape:", scores.shape)

# ver cada head
for i in range(scores.shape[1]):
    print(f"\nHead {i}")
    print(scores[0, i])
```

tf and hf

TensorFlow → é um *framework de machine learning* (base)

1. Como usar TensorFlow

1.1 Instalar

```
pip install tensorflow
```

1.2 como usar num programa

```
Import tensorflow as tf
```

Hugging Face (transformers) → é uma *biblioteca de modelos prontos* (alto nível)

1. Como usar Hugging Face (Transformers)

1.1 Instalar

```
pip install transformers
```

1.2 como usar num programa

Clasificador de sentimentos para texto

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
print(classifier("I am your friend"))
```

1.3. Usar Hugging Face de forma mais “avançada”

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased")
inputs = tokenizer("Hello world", return_tensors="pt")
outputs = model(**inputs) # *-desempacota list, **-desempacota dicionários
# [ 1, 29, “teste”] desempacotar transforma em argumentos 1, 29, “teste”
```

1.4. Usar Hugging Face com TensorFlow

```
from transformers import TFAutoModel
model = TFAutoModel.from_pretrained("bert-base-uncased")
```

Componentes principais do HF

Componentes principais da biblioteca Transformers (Hugging Face)					
Categoria	Classe / Função	Para que serve	Pertence a	Executado por baixo	Exemplo
Alto nível	pipeline	Usar modelos facilmente	HF	PyTorch (default) ou TensorFlow	pipeline("sentiment-analysis")("I love this")
Tokenização	AutoTokenizer	Converter texto em tokens	HF	HF (Python)	tokenizer("Hello", return_tensors="pt")
Modelo base	AutoModel	Gerar embeddings	HF	PyTorch	model(**inputs)
Classificação	AutoModelForSequenceClassification	Classificar texto	HF	PyTorch	model(**inputs).logits
Geração	AutoModelForCausalLM	Gerar texto	HF	PyTorch	model.generate(**inputs)
Seq2Seq	AutoModelForSeq2SeqLM	Tradução / resumo	HF	PyTorch	model.generate(**inputs)
QA	AutoModelForQuestionAnswering	Perguntas e respostas	HF	PyTorch	model(**inputs).start_logits
Treino	Trainer	Treinar modelos	HF	PyTorch	trainer.train()
Config treino	TrainingArguments	Definir parâmetros de treino	HF	HF	TrainingArguments(output_dir="./out")
Dados	load_dataset	Carregar datasets	HF	HF	load_dataset("imdb")

Componentes principais do HF (Interpretação da Tabela)

Descrição da tabela
Esta tabela organiza os principais componentes da biblioteca Transformers por função no fluxo de trabalho de machine learning.

1. Estrutura geral

Cada linha representa:

- uma classe ou função importante
- agrupada por categoria (o papel que desempenha)

2. Como ler cada coluna

- Categoria → o papel no sistema (ex: tokenização, treino, etc.)
- Classe / Função → nome que usas no código
- Para que serve → função principal
- Pertence a → biblioteca onde está definida (sempre HF)
- Executado por baixo → quem faz o cálculo real
- Exemplo → uso típico em código

3. Fluxo completo (muito importante)

A tabela segue o fluxo típico de uso:

Texto → Tokenizer → Modelo → Output

↑

Dataset

↓

Trainer

4. Ideia principal

- A Hugging Face fornece todas estas classes (interface)
- O trabalho pesado é feito por:
  - PyTorch (mais comum)
  - TensorFlow (opcional)

Conclusão

Esta tabela resume os componentes essenciais para:

- usar modelos
- preparar dados
- treinar
- gerar resultados

Transformers no Tensorflow

```python
import tensorflow as tf
from tensorflow.keras import layers

class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()

        # Multi-head attention
        self.att = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)

        # Feed-forward network (FFN)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])

        # Normalização
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)

        # Dropout (opcional)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)
```

```python
    def call(self, inputs, training=False):
        # Atenção
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)

        # FFN
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)

        return self.layernorm2(out1 + ffn_output)
```

Transformer block faz:

input
↓
attention
↓ + residual norm
↓
FFN
↓ + residual norm

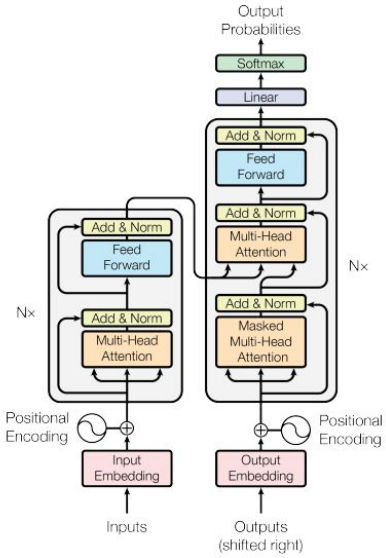

Figure 1: The Transformer - model architecture.

Como usar o Block

```
inputs = tf.keras.Input(shape=(None, 64))  # sequência com embedding de 64
x = TransformerBlock(embed_dim=64, num_heads=4, ff_dim=128)(inputs)
outputs = layers.Dense(10, activation="softmax")(x)
model = tf.keras.Model(inputs=inputs, outputs=outputs)
model.summary()
```

Neste exexmplo os “3” (Q, K, V):

- não são argumentos do construtor
- são criados automaticamente dentro da layer
- aparecem como inputs na chamada (`query, key, value`)

há **3 transformações diferentes do mesmo input**

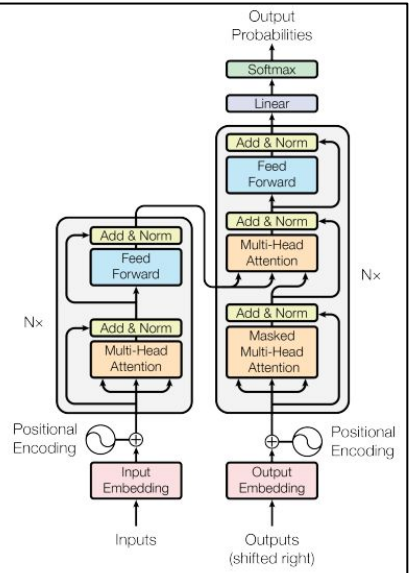

Figure 1: The Transformer - model architecture.

O que este código tem (ligando à teoria)

- MultiHeadAttention → atenção (core do Transformer)
- Dense → camadas FC (FFN)
- LayerNormalization → estabiliza treino
- Dropout → evita overfitting
- Residual connections → inputs + attn_output

Ou seja: isto é exatamente o bloco do paper **“Attention Is All You Need”**

FIM